

Assignment 7

ART Image Compression and Reconstruction

Connor Prikkel

ECE 550: Artificial Neural Networks

22 January 2026



**University
of Dayton**

Contents

1 Introduction	3
2 Methodology	3
2.1 Background	3
2.2 Program Flowchart	3
3 Results	4
3.1 Procedure	4
3.2 Evaluation Criteria	5
3.3 Visualization	5
4 Conclusion	9

1 Introduction

In this homework, the student repurposes the previously implemented Adaptive Resonance Theory (ART) neural network for use in image compression and reconstruction.

2 Methodology

2.1 Background

First, an arbitrary input image is loaded and preprocessed. This involves normalizing the input and dividing the image into blocks that fill the entire space but are not overlapping. Each block is then flattened into a vector of pixel values. These vectors are fed into the ARTNN, which iteratively compares each input vector with the existing clusters. An example visual representation of these blocks is given below in Figure 1.

0.03 0.13 0.36 0.22	0.56 0.84 0.67 0.29	0.03 0.26 0.36 0.22	0.56 0.84 0.97 0.29
0.76 0.23 0.36 0.22	0.33 0.44 0.67 0.29	0.73 0.23 0.86 0.22	0.56 0.84 0.91 0.29
0.19 0.33 0.36 0.22	0.56 0.04 0.67 0.29	0.21 0.21 0.21 0.21	0.56 0.84 0.67 0.29
0.16 0.46 0.36 0.22	0.56 0.64 0.21 0.21	0.21 0.21 0.22 0.21	0.56 0.54 0.67 0.29

Figure 1: Block Representation with Minimum Pixel Values

This is an $M \times N$ representation where $M = 4$ columns and $N = 4$ rows for a total of 16 blocks. Each block contains the normalized pixel values for that region of the image. In this example, the block size is 2×2 . The choice function, vigilance criteria, and other steps of the ART are followed as before but for each and every block. Each block is then given a unique cluster index depending on its membership. Once all blocks are clustered, the process is repeated until no new clusters are created or until a max number of iterations is completed if the clusters do not converge.

The cluster sequences are then compressed through run-length encoding (RLE), a data processing technique which replaces any consecutive, repeated values in the sequence with a single value and a corresponding count of how many times it appears in a row. Run-length decoding (RLD) then reconstructs the original data sequence from its compressed version. The image is then reconstructed from this sequence.

2.2 Program Flowchart

The following steps define the compression and reconstruction process used in conjunction with the ARTNN. Any equations referenced can be viewed in the last report and are not repeated here for the sake of brevity.

1) Load in and resize/normalize input image x_i

|

2) Split x_i into a 64×64 grid of blocks with block size (4,4)

|

3) Compute the choice function CF_j

|

4) Determine the winner node CF_J

|

5) Compute the vigilance function VF_J

|

6) Determine whether VF_J passes or fails against the ρ

|

6a) IF $VF_J \geq \rho \Rightarrow$ pass!
Update the winning cluster's weights

|

6b) IF $VF_J < \rho \Rightarrow$ fail!
Reset $CF_i = 0$, find the next winner as outlined in Step (3)

|

7) IF no existing clusters satisfy the vigilance test, create a new cluster

|

8) Repeat Steps (3) - (7) for each block

|

9) Repeat Steps (3) - (8) until no new clusters are created and the outputs stabilize OR the maximum number of iterations is reached

3 Results

3.1 Procedure

A low, medium, and high complexity image are loaded into PyTorch from a free open-source library named 'scikit'. The low complexity image is a clock, the medium complexity image is an eagle upon a perch, and the high complexity image features a cameraman. The ART network from the last project is modified to support this specific application. Additional methods to split each image into blocks, perform RLE and RLD, and ultimately reconstruct the image are also defined in Python. The network is then trained and tested on each of these images.

3.2 Evaluation Criteria

Each of the reconstructed images is evaluated against the originals utilizing three distinct evaluation metrics: compression ratio, mean-squared error (MSE) and peak signal-to-noise (PSNR) ratio. The compression ratio acts a measure of how efficiently the data has been compressed and is defined by Equation 1:

$$CR = \frac{O_s}{C_s + RLE_s} \quad (1)$$

where O_s is the number of pixels in the original image, C_s is the size of the codebook, or mapping of clusters to weights, and RLE_s is the size of the cluster indices produced by the ARTNN and after run-length-encoding. A higher CR indicates better compression. MSE is a measure of “closeness” of corresponding pixels between the original and reconstructed images. It is given by Equation 2 below:

$$MSE = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (P_{o-ij} - P_{r-ij})^2 \quad (2)$$

where P_{o-ij} is the intensity value of a certain pixel in the original image and P_{r-ij} is the intensity value of the same pixel in the reconstructed image. A higher MSE means the reconstruction is farther from matching the original image and vice versa for a lower MSE. If the $MSE = 0$, the reconstructed image is identical to the original. Finally, PSNR compares the highest intensity pixel against its MSE value and can be computed by:

$$PSNR = 10 \log_{10} \left(\frac{MAX}{MSE} \right) \quad (3)$$

where MAX is the maximum intensity value of the reconstructed image. A higher PSNR indicates higher quality reconstruction. Each of these values is calculated for each reconstructed image and summarized in Table 1.

Image Complexity	CR	MSE	PSNR (dB)
Low	52.4288	0.0021	26.68
Medium	3.01260	0.0455	13.42
High	3.6232	0.0153	18.15

Table 1: Reconstructed Image Evaluation Metrics

The clock image has the best compression ratio by far as it has the most repeated clusters and the fewest switches in pixel intensities across the entire image. The MSE remains fairly low across all images, which is indicative that the reconstructed image is quantitatively close to the original in all cases. The PSNR is lower than desired for the medium and high images, though this is likely due to the fact that the ART network did not converge for either of these images. Both of these images are ran for only 30 iterations as running them for more causes the program to get stuck for a very long period of time in the training loop. This occurs as the number of clusters for both of these images is much larger so when every new block is iterated through it is compared against a larger and larger number of existing clusters.

3.3 Visualization

Heatmaps revealing the ART's cluster mappings for every block of each image are visualized below in Figure 2, Figure 3, and Figure 4.

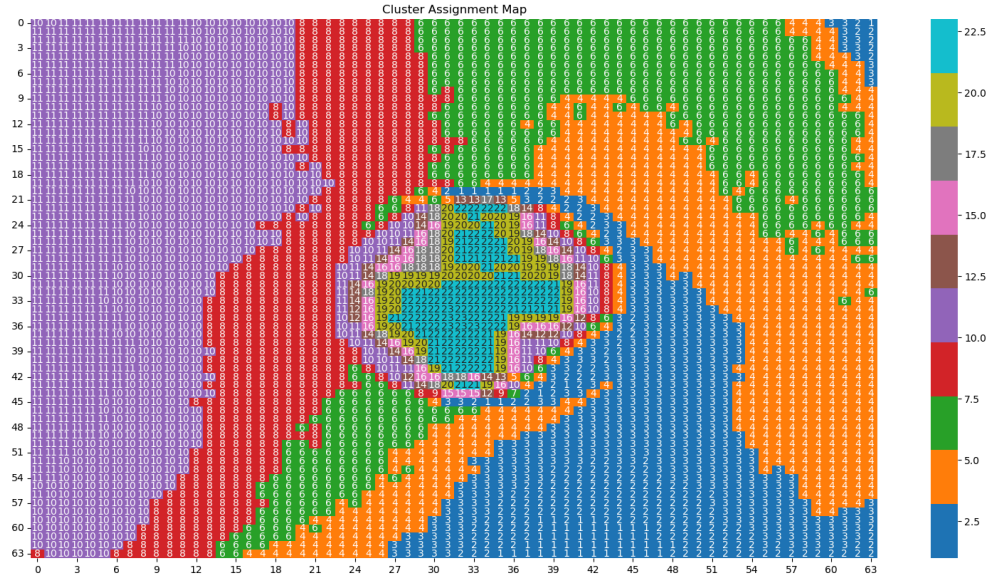


Figure 2: Low Complexity Cluster Map

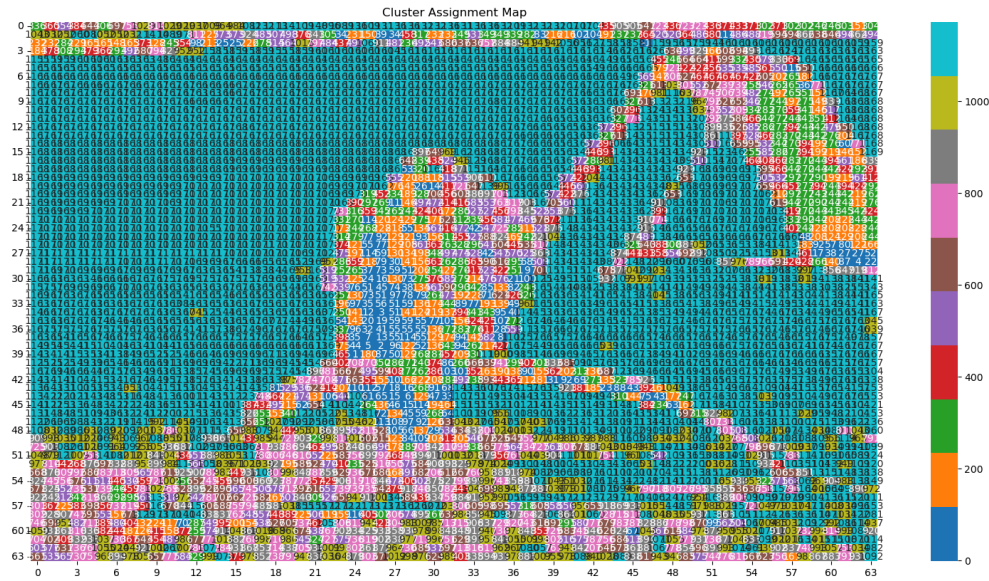


Figure 3: Medium Complexity Cluster Map

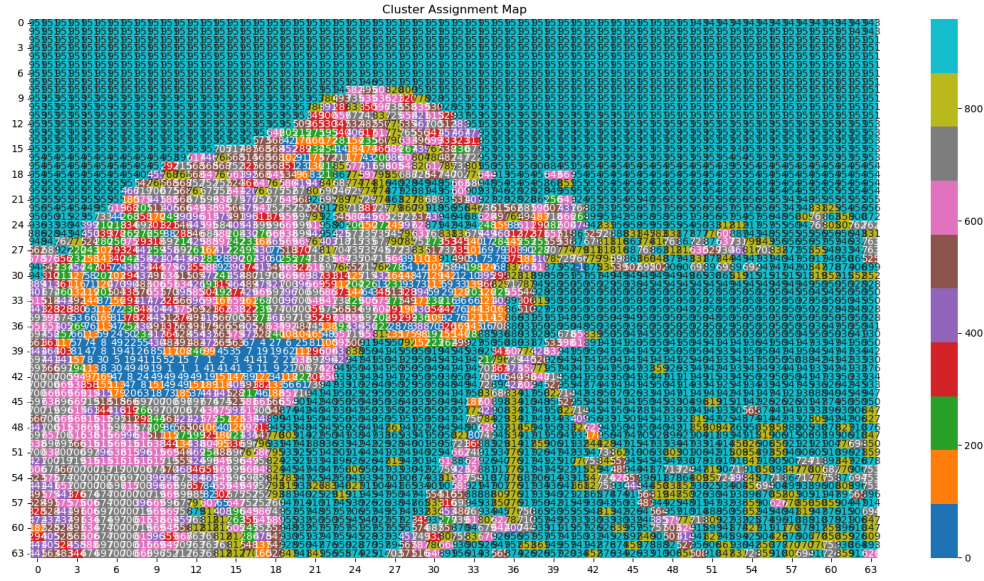


Figure 4: High Complexity Cluster Map

Every original image is then plotted with its reconstruction below in Figure 5, Figure 6, and Figure 7.

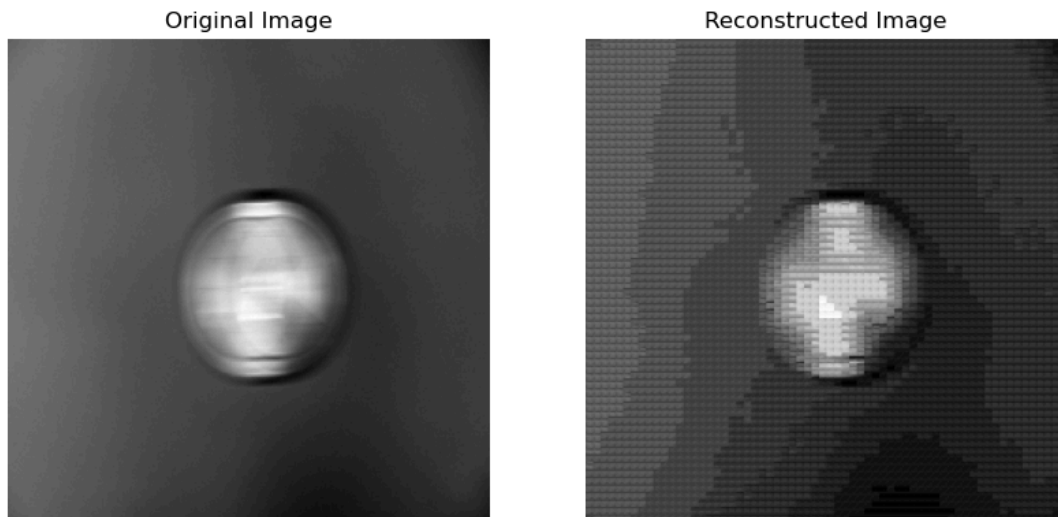


Figure 5: Low Complexity Images

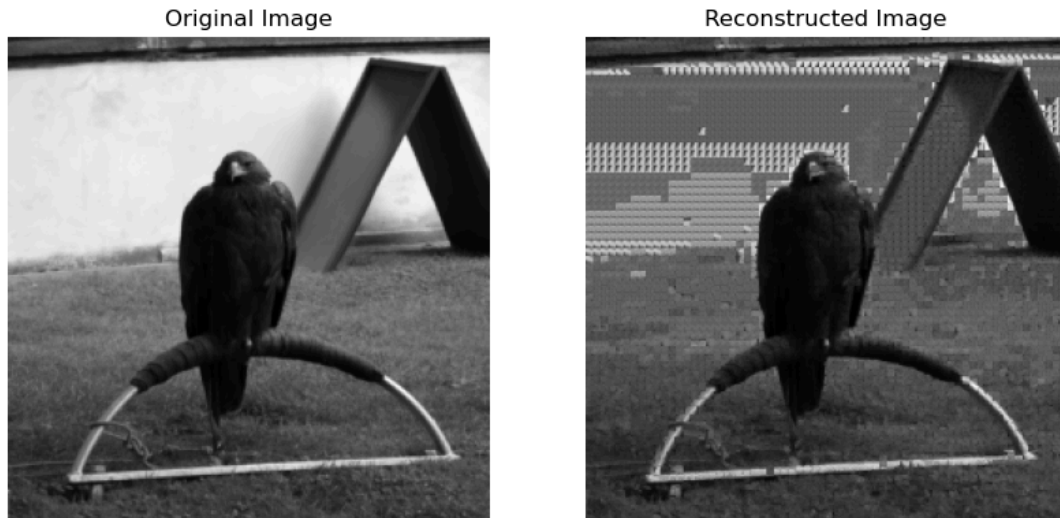


Figure 6: Medium Complexity Images

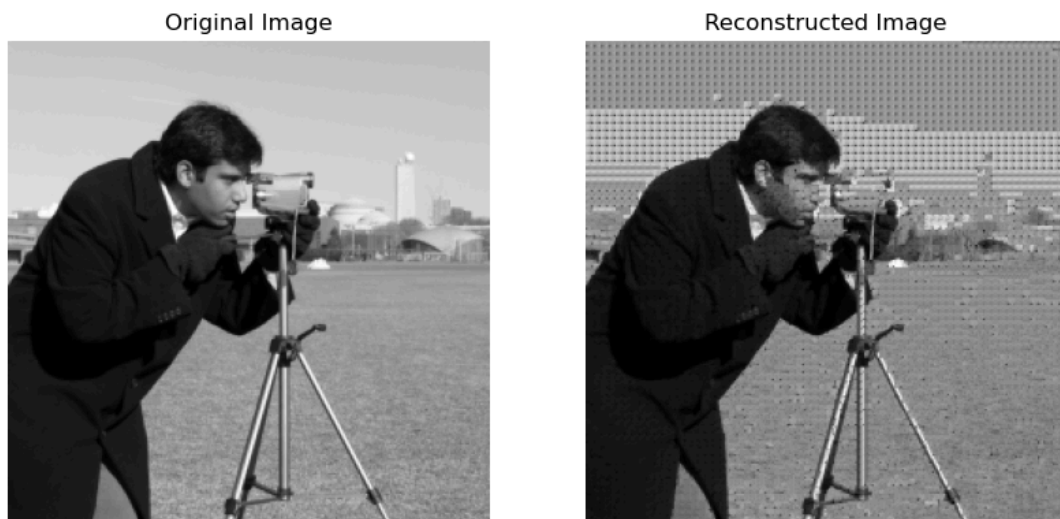


Figure 7: High Complexity Images

Figure 5 reveals that the reconstructed low complexity image is very akin to the original aside from appearing slightly granular. Figure 6 and Figure 7 illustrate decent reconstructions of the eagle and cameraman but feature bad artefacts in the background. A potential remedy to this is training for longer or with a better CPU/GPU.

4 Conclusion

A practical real-world application of the ART neural network in image compression and reconstruction is realized in PyTorch. The advantage of an unsupervised learning network for this task is readily apparent: it can efficiently classify and organize pixels into clusters where other supervised learning networks could struggle. However, the limitation of this network becomes apparent with the training on the medium and high complexity images. A higher ρ increases the selectivity of this network, improving accuracy but incurring a computational cost. As discussed in the previous project, this parameter should be carefully chosen.